



# CSCI 112

## Programming with C

### LECTURE: 5

J. L. Pach

# OUTLINE:

Control flow:

✂ if-else statement

✂ switch statement

✂ goto statement

# CONTROL

**Flow**

# IF-ELSE STATEMENT 1/4

```
if (expression1)
    single_statement1
else
    single_statement2
```

- ⚡ `expression1` is non-zero  $\Rightarrow$  `statement1`
- ⚡ `expression1` could be series of expressions
- ⚡ Block can be substitute for `simple_statement`
- ⚡ `else` is optional

## IF-ELSE STATEMENT 2/4

```
int main()
{
    int x = 5;
    if( x > 1 )
        printf("%s\n", "a variable x is greater than 1");
}
```



a variable x is greater than 1

- ✂ Why is an if-else statement considered a statement when it doesn't change anything on its own?
- ✂ It's not true that it doesn't change the PC, but to explain that we need to understand ternary operator and labels.

## IF-ELSE STATEMENT 3/4

✂ With optional `else`

```
int main()
{
    int x = 5;
    if( x > 1 )
        printf("%s\n", "a variable x is greater than 1");
    else
        printf("%s\n", "a variable x is NOT greater than 1");
}
```



a variable x is greater than 1

## IF-ELSE STATEMENT 4/4

```
int main()
{
    int x = 0;
    if( x > 1 )
    {
        printf("%s", "a variable x");
        printf("%s\n", " is greater than 1");
    }
    else
    {
        printf("%s", "a variable x");
        printf("%s\n", " is not greater than 1");
    }
}
```



a variable x is not greater than 1

## NESTED IF-ELSE STATEMENT 1/3

- ✂ Since an `if` statement following its condition in parentheses (the expression that, if it evaluates to a non-zero value, executes the subsequent code block) is considered a statement itself;
- ✂ it's important to note that another if statement can be placed after either the code block of the first `if` or its `else` block (if present).

## NESTED IF-ELSE STATEMENT 2/3

```
int main()
{
    int x = 1;

    if(x == 0)
    ;
    else if(x == 1)
    ;
    else if(x > 2)
    ;
    else
    ;
}
```



```
int main()
{
    int x = 1;
    if(x == 0)
    {
    }
    else
    if(x == 1)
    {
    }
    else
    if(x > 2)
    {
    }
    else
    {
    }
}
```



# NESTED IF-ELSE STATEMENT 3/3

```
int main()
{
    int x = getche() - 48;
    printf("%s", "a variable x is ");

    if(x == 0)
    {
        printf("%s\n", "0");
    }
    else
    {
        if(x == 1)
        {
            printf("%s\n", "1");
        }
        else
        {
            if(x > 1 && x < 5)
            {
                printf("%s\n", "between 2 and 4");
            }
            else
            {
                printf("%s\n", "not between 0 and 4");
            }
        }
    }
}
```



```
int main()
{
    int x = getche() - 48;
    printf("%s", "a variable x is ");

    if(x > 0)
    if(x < 5)
        printf("%s\n", "between 1 and 4");

}
```



# SWITCH

**statement**

# SWITCH STATEMENT 1/1

```
switch (expression)
{
    case <constant-expression1>:
        /* code */
        break;

    case <constant-expression2>:
        /* code */
        break;

    default:
        /* code */
        break;
}
```



```
int main()
{
    int x = getche() - 48 ; // typed 4
    printf("%s", "a variable x is ");
    switch (x) /* constant-expression */
    {
        case 1:
            printf("%s\n", "1");
            break;
        case 2:
            printf("%s\n", "2");
            break;
        default:
            printf("%s\n", "not between 1 and 3"); // <- true
            break;
    }
} // result: "a variable x is not between 1 and 3"
```

## SWITCH STATEMENT 1/2

In C programming, a **constant expression** is an expression that evaluates to a fixed value at compile time, meaning the value is determined during the compilation process rather than during program execution.

- ✂ The `default` block in the switch statement is optional!
- ✂ The `default` block can be used for error handling.
- ✂ It's important to note that omitting the `default` block will cause the program to skip the entire switch statement if the variable takes on a value not defined in the switch statement.
- ✂ To terminate a switch statement, we use the `break` keyword

# SWITCH STATEMENT 1/3

```
int main()
{
    int x = getche() - 48; //typed 2
    printf("%s", "a variable x is ");
    switch (x) /* constant-expression */
    {
        case 1:
            printf("%s\n", "1");
            break;
        case 2:
            printf("%s\n", "2");
            break;
    }
} // result: "a variable x is 2"
```



# SWITCH STATEMENT 1/4

```
int main()
{
    int x = getche() - 48; // typed 2
    printf("%s", "a variable x is ");
    switch (x) /* constant-expression */
    {
        case 1:
        case 2:
        case 3:
            printf("%s\n", "between 1 and 3");
            break;
        default:
            printf("%s\n", "not between 1 and 3");
            break;
    }
} // result: "a variable x is 1 and 3"
```



```
int main()
{
    int x = getche() - 48; // typed 2
    printf("%s", "a variable x is ");
    switch (x) /* constant-expression */
    {
        case 1:
            printf("%s", "between");
        case 2:
            printf("%s", " 1 and");
        case 3:
            printf("%s\n", " 3");
            break;
        default:
            printf("%s\n", "not between 1 and 3");
            break;
    }
} // result: "a variable x is 1 and 3"
```



# DIFFERENCE BETWEEN SWITCH & IF-ELSE

✂ **if** statements:

✂ **Dynamic Conditions:** if statements allow for dynamic conditions that can change during program execution. These conditions can involve complex expressions, function calls, and even user input.

✂ **switch** statements:

✂ **Static Values:** switch statements are primarily designed for evaluating expressions against a set of predefined, static values. These values are typically determined before program execution.

# GOTO

**statement**

# GOTO STATEMENT 1/2

```
int main()
{
    bool is_true = true;
    if ( is_true )
        goto label1;
    else
        goto label2;

    label1:
        printf("It was true\n");
        goto end;
    label2:
        printf("It was false\n");
        goto end;
    end:
} // result: It was true
```



## GOTO STATEMENT 2/2

- ✂ goto allows us to jump (within the same function) unconditionally to a location identified by a `label`
- ✂ a simple example of Spaghetti Code - Dijkstra, E. W. (1968)

# TERNARY

**conditional operator**

# TERNARY CONDITIONAL OPERATOR?:

- ✂ If `expression1` evaluates to `true`, the ternary operator returns the value of `expression2`. Otherwise, it returns the value of `expression3`.

```
<expression1> ? <expression2> : <expression3>;
```

```
int main()
{
    int i = 1;
    i = (i > 0) ? 2 : 3;    /* ternary conditional */
    printf("%d\n", i);
}
```



Result:

```
2
```



# IF-ELSE STATEMENT

```

/* 01 */  int main()          /* Program Counter =01;      PC++; “goto” new PC */
/* 02 */  {                   /* Program Counter =02;      PC++; “goto” new PC */
/* 03 */      if ( x > 0 )     /* if x > 0 then PC = 04 otherwise PC = 10; “goto” new PC */
/* 04 */  {                   /* Program Counter =04;      PC++; “goto” new PC */
/* 05 */      ;               /* Program Counter =05;      PC++; “goto” new PC */
/* 06 */      ;               /* Program Counter =06;      PC++; “goto” new PC */
/* 07 */      ;               /* Program Counter =07;      PC++; “goto” new PC */
/* 08 */  }                   /* Program Counter =08;      “goto” PC=15 */
/* 09 */      else           /*                               */
/* 10 */  {                   /* Program Counter =10;      PC++; “goto” new PC */
/* 11 */      ;               /* Program Counter =11;      PC++; “goto” new PC */
/* 12 */      ;               /* Program Counter =12;      PC++; “goto” new PC */
/* 13 */      ;               /* Program Counter =13;      PC++; “goto” new PC */
/* 14 */  }                   /* Program Counter =14;      PC++; “goto” new PC */
/* 15 */  }                   /* Program Counter =15;      PC++; “goto” new PC */

```

# MAGIC...

```
int main()
{
/* 01 */      int x = 10;
/* 02 */      int * label_pointer;
/* 03 */      label_pointer = ( x > 0 ) ? &&label_greater: &&label_lower; /* equals 08 or 12 */
/* 04 */      goto *label_pointer;
/* 05 */      label_greater:
/* 06 */          printf("%s\n", "a variable x is greater than 0");
/* 07 */          goto end_label;
/* 08 */      label_lower:
/* 09 */          printf("%s\n", "a variable x is lower than 0");
/* 11 */          goto end_label;
/* 12 */      end_label:
/* 13 */          ;
}
```



THANK

*You*